

LAB5

Name : P.Dinesh Manideep

Roll No : 231IT047

Execute the MD5 Algorithm for the plain text of Assignment of IAS.

Learn the conversion from plaintext to MD5 and justify the answer for the plain texts

a) Computer Networks is a prerequisite

b) Networking is full of Attacks

Steps of the MD5 Algorithm

1. Convert Plaintext to Binary

The input message (plaintext) is first converted into a sequence of binary bits using ASCII or UTF-8 encoding. Each character becomes 8 bits, forming the initial bit stream that MD5 will process.

2. Message Padding

A single '1' bit is appended to the message, followed by '0' bits until the total length becomes $448 \bmod 512$ bits. Padding ensures that the message length fits the block size required by the MD5 algorithm.

3. Append Original Message Length

After padding, the 64-bit representation of the original message length (before padding) is added to the end of the message. This makes the total message size a multiple of 512 bits.

4. Initialize MD5 Buffers

MD5 initializes four 32-bit registers (buffers) that hold intermediate values during hashing:

- A = 0x67452301
- B = 0xEFCDAB89
- C = 0x98BADCFE
- D = 0x10325476

These registers are updated during the processing rounds.

5. Divide Message into 512-bit Blocks

The padded message is divided into blocks of 512 bits. Each block is further split into 16 words of 32 bits each. The algorithm processes one block at a time.

6. Perform Four Rounds of Hash Computation

Each block goes through 64 operations divided into 4 rounds. Each round uses a different logical function:

- Round 1: $F(B,C,D) = (B \text{ AND } C) \text{ OR } (\sim B \text{ AND } D)$
- Round 2: $G(B,C,D) = (B \text{ AND } D) \text{ OR } (C \text{ AND } \sim D)$
- Round 3: $H(B,C,D) = B \text{ XOR } C \text{ XOR } D$
- Round 4: $I(B,C,D) = C \text{ XOR } (B \text{ OR } \sim D)$

These operations combine the message block, constants, and buffer values with bitwise operations and circular shifts.

- Each step performs the operation:
- $A = B + \text{LeftRotate}(A + f(B,C,D) + M[g] + K[i], s)$
- where
 - $f = F, G, H, \text{ or } I$ depending on the round
 - $M[g]$ = message word
 - $K[i]$ = constant value
 - s = shift amount
- Constant generation:
- $K[i] = \text{floor}(2^{32} \times |\sin(i+1)|)$, for $i = 0 \dots 63$
- Left rotation:
- $\text{LeftRotate}(x,s) = (x \ll s) \text{ OR } (x \gg (32 - s))$

7. Update Buffer Values

After completing the 64 operations for a block, the results are added to the previous buffer values (A, B, C, D). This updated state is then used for the next message block.

8. Produce Final Hash Value

After all blocks are processed, the final values of A, B, C, and D are concatenated together to form a 128-bit MD5 hash. This hash is usually displayed as a 32-character hexadecimal string.

Implementation of MD5 in python

Code :

```
import math
import struct

# Left rotate operation
def left_rotate(x, c):
    return ((x << c) | (x >> (32 - c))) & 0xFFFFFFFF

class MD5:

    def __init__(self, message):
        self.message = message.encode()
        self.init_buffers()

    # Step 1: Initialize buffers
    def init_buffers(self):
        self.A = 0x67452301
        self.B = 0xEFCDAB89
        self.C = 0x98BADCFE
        self.D = 0x10325476

    # Step 2: Padding
    def pad_message(self):

        msg = bytearray(self.message)
        orig_len_bits = (8 * len(msg)) & 0xffffffff

        # append 1 bit
        msg.append(0x80)

        # append zeros
        while (len(msg) * 8) % 512 != 448:
```

```

        msg.append(0)

# append length
msg += struct.pack('<Q', orig_len_bits)

return msg

# Step 3: Process blocks
def process(self):

    msg = self.pad_message()

    # Shift amounts
    s = [
        7,12,17,22,7,12,17,22,7,12,17,22,7,12,17,22,
        5,9,14,20,5,9,14,20,5,9,14,20,5,9,14,20,
        4,11,16,23,4,11,16,23,4,11,16,23,4,11,16,23,
        6,10,15,21,6,10,15,21,6,10,15,21,6,10,15,21
    ]

    # Constants
    K = [int(abs(math.sin(i + 1)) * (2**32)) & 0xFFFFFFFF for i in range(64)]

    for offset in range(0, len(msg), 64):

        chunk = msg[offset:offset+64]
        M = list(struct.unpack('<16I', chunk))

        A, B, C, D = self.A, self.B, self.C, self.D

        for i in range(64):

            if i < 16:
                F = (B & C) | (~B & D)
                g = i

            elif i < 32:
                F = (D & B) | (~D & C)
                g = (5*i + 1) % 16

            elif i < 48:
                F = B ^ C ^ D
                g = (3*i + 5) % 16

```

```
else:
```

```
    F = C ^ (B | ~D)
```

```
    g = (7*i) % 16
```

```
    temp = (A + F + K[i] + M[g]) & 0xFFFFFFFF
```

```
    A, D, C, B = D, C, B, (B + left_rotate(temp, s[i])) & 0xFFFFFFFF
```

```
# Add this chunk's hash
```

```
self.A = (self.A + A) & 0xFFFFFFFF
```

```
self.B = (self.B + B) & 0xFFFFFFFF
```

```
self.C = (self.C + C) & 0xFFFFFFFF
```

```
self.D = (self.D + D) & 0xFFFFFFFF
```

```
# Step 4: Produce final hash
```

```
def hexdigest(self):
```

```
    self.process()
```

```
    return ''.join(f'{x:02x}' for x in struct.pack('<4l', self.A, self.B, self.C, self.D))
```

```
# Test the plaintexts
```

```
text1 = "Assignment of IAS"
```

```
text2 = "Computer Networks is a prerequisite"
```

```
text3 = "Networking is full of Attacks"
```

```
print("Plaintext 1:", text1)
```

```
print("MD5 Hash:", MD5(text1).hexdigest())
```

```
print()
```

```
print("Plaintext 2:", text2)
```

```
print("MD5 Hash:", MD5(text2).hexdigest())
```

```
print()
```

```
print("Plaintext 3:", text3)
```

```
print("MD5 Hash:", MD5(text3).hexdigest())
```

```
• [dinesh@dinesh lab_5]$ python md5_implimentation.py
Plaintext 1: Assignment of IAS
MD5 Hash: 32f7de05cf559273a7af18f20a885a3c

Plaintext 2: Computer Networks is a prerequisite
MD5 Hash: 17953e4922d8fd0f17a7fc3ea717de60

Plaintext 3: Networking is full of Attacks
MD5 Hash: 7028a08ac019244417deb997553aaca7
❖ [dinesh@dinesh lab_5]$
```

Implementation using library

```
import hashlib
```

```
# Plain texts
```

```
text1 = "Assignment of IAS"
```

```
text2 = "Computer Networks is a prerequisite"
```

```
text3 = "Networking is full of Attacks"
```

```
# Convert to bytes and compute MD5
```

```
hash1 = hashlib.md5(text1.encode()).hexdigest()
```

```
hash2 = hashlib.md5(text2.encode()).hexdigest()
```

```
hash3 = hashlib.md5(text3.encode()).hexdigest()
```

```
print("Plaintext 1:", text1)
```

```
print("MD5 Hash:", hash1)
```

```
print()
```

```
print("Plaintext 2:", text2)
```

```
print("MD5 Hash:", hash2)
```

```
print()
```

```
print("Plaintext 3:", text3)
```

```
print("MD5 Hash:", hash3)
```

```
• [dinesh@dinesh lab_5]$ python md5_withlib.py
Plaintext 1: Assignment of IAS
MD5 Hash: 32f7de05cf559273a7af18f20a885a3c

Plaintext 2: Computer Networks is a prerequisite
MD5 Hash: 17953e4922d8fd0f17a7fc3ea717de60

Plaintext 3: Networking is full of Attacks
MD5 Hash: 7028a08ac019244417deb997553aaca7
❖ [dinesh@dinesh lab_5]$
```

we can see the outputs are same with the library functions as with the implementation of the MD5 algorithm

Justification

The MD5 algorithm was implemented in Python by following all the required steps such as padding the message, adding the message length, initializing buffers, performing the four rounds of operations, and generating the final hash. The same plaintext was also hashed using Python's **hashlib MD5 library**. The hash values obtained from both methods were the same. This confirms that the implementation of the MD5 algorithm is correct.